NLP (CS60075)

Autumn Semester 2026

Assignment 2

Project 10 — Open-Domain Question Answering: From Lexical Retrieval to RAG

Dataset 10.3 — TriviaQA (`rc.nocontext`)

Group No.: 30

Group Members:

1. B. Siddaarth (23CS30011)
2. Bala Dharnesh (23CS10008)
3. Subham Ghosh (22EE38015)

Contents

1 Introduction **3**

2 Task 1 — Baseline: Classical Lexical Retrieve-and-Extract **3**
2.1 23CS30011 — B. Siddaarth . . . 3
2.1.1 Method . . . 3
2.1.2 Quantitative Results . . . 4
2.1.3 Qualitative Analysis . . . 5
2.1.4 What Did Not Work / Limitations . . . 5
2.2 23CS10008 — Bala Dharnesh . . . 5
2.2.1 Method . . . 5
2.2.2 Quantitative Results . . . 5
2.3 22EE38015 — Subham Ghosh . . . 6
2.3.1 Method . . . 6
2.3.2 Quantitative Results . . . 6
2.4 Comparison Across the Three Task 1 Implementations . . . 6

3 Task 2 — Paper Reproduction: Retrieval-Augmented Generation **7**
3.1 23CS30011 — B. Siddaarth . . . 7
3.1.1 Method . . . 7
3.1.2 Quantitative Results . . . 8
3.1.3 Qualitative Analysis . . . 9
3.1.4 Paper-Reproduction Comparison . . . 9
3.2 23CS10008 — Bala Dharnesh . . . 10
3.2.1 Method . . . 10
3.2.2 Quantitative Results . . . 10
3.3 22EE38015 — Subham Ghosh . . . 11
3.3.1 Method . . . 11
3.3.2 Quantitative Results . . . 11
3.3.3 Qualitative Analysis . . . 11
3.4 Comparison Across the Three Task 2 Implementations . . . 11

4 Task 3 — Combine & Improve: Hybrid Retrieval with Groundedness Verification **12**
4.1 23CS30011 — B. Siddaarth . . . 12
4.1.1 Method . . . 12
4.1.2 Quantitative Results . . . 13
4.1.3 Qualitative Analysis . . . 14
4.1.4 Why the Correction Experiments Failed . . . 15
4.2 23CS10008 — Bala Dharnesh . . . 16
4.2.1 Method . . . 16
4.2.2 Quantitative Results . . . 16
4.3 22EE38015 — Subham Ghosh . . . 17
4.3.1 Method . . . 17
4.3.2 Quantitative Results . . . 17
4.3.3 Qualitative Analysis . . . 18
4.4 Comparison Across the Three Task 3 Implementations . . . 18

5 Discussion **18**

1 Introduction

This report covers all three tasks of Project 10 on our assigned dataset, TriviaQA `rc.nocontext` [1]: Task 1 builds a fully classical, non-neural retrieve-and-extract pipeline; Task 2 reproduces the retrieval-augmented generation idea from Lewis et al. [2]; and Task 3 combines the retrieval signals from Tasks 1 and 2 and adds a groundedness check on top of Task 2's generations.

This is a group submission. All three of us — B. Siddaarth (23CS30011), Bala Dharnesh (23CS10008), and Subham Ghosh (22EE38015) — independently designed and implemented all three tasks end to end, rather than splitting the assignment by task. We did this deliberately: with three genuinely different implementations of the same three tasks (different corpus-construction procedures, different generators, different code architectures, and in one case a different underlying design philosophy for the retrieval metric itself), comparing them side by side is more informative than any one of us working alone, and it let us sanity-check our own results against two independently-built pipelines rather than trusting a single implementation. We did not merge our code into one shared pipeline — each person's implementation is kept intact and submitted as written, under `code/<task>/<roll_no>_task<n>/` (e.g. `code/task1/23cs30011_task1/`), with each person's complete original project additionally kept as one unit under `code/<roll_no>_project/` for anyone who wants to run a submission's full pipeline end to end (see `README.md` for exactly how each person's code is organised and run).

This report mirrors that structure: each of the three Task sections below is split into one subsection per author, covering that author's method, quantitative results, and qualitative analysis, followed by a short subsection comparing all three approaches on the same task. The section order and subsection headings are kept identical across all three tasks so the same comparison is easy to follow throughout. All three implementations target the same dataset (TriviaQA `rc.nocontext` [1], validation split, 300 fixed questions, seed 42) and the same architectural brief (Task 1: BM25 + heuristic extraction; Task 2: dense retrieval + an open generator, no fine-tuning; Task 3: fuse Task 1 and Task 2's retrieval signals and add a groundedness check) but arrive at it via three independently-built Wikipedia corpora, three different generator choices, and, in places, different reasonable readings of an underspecified part of the brief (most notably Dataset 10.3's missing corpus-construction section, discussed separately by each author below).

Environment setup, exact run commands, and where every output file gets written are documented in `README.md` for all three submissions; we do not repeat that here and instead focus this report on method, results, and discussion, as the assignment brief asks for.

2 Task 1 — Baseline: Classical Lexical Retrieve-and-Extract

2.1 23CS30011 — B. Siddaarth

2.1.1 Method

Sampling and index construction. We load the `validation` split of `mandarjoshi/trivia_qa`, config `rc.nocontext` (17,944 questions), and draw a fixed sample of 300 with seed 42. This config intentionally removes the `entity_pages` and `search_results` fields that the full `rc` config bundles, leaving only the bare question and

the gold answer (a `value` string plus a list of `aliases`) — so there is no passage corpus to retrieve from unless we build one ourselves.

To build one, we lean on a property of TriviaQA: the great majority of its answers are themselves Wikipedia-entity strings (`answer['type'] == 'WikipediaEntity'`). For each of the 300 sampled questions we take `answer['value']`, resolve it to a Wikipedia page title via the MediaWiki `opensearch` API (a fuzzy title search, not full-text search), and fetch that page's plain-text extract. This only decides *which page* belongs in the index — it is not handed to the retriever or the answer extractor, and all 284 resulting pages are pooled into one shared corpus, so a question's BM25 search still has to find the right page among roughly 300 unrelated distractors. Of 294 unique answer strings, 285 resolved on the first try; a small, deterministic normalisation pass (strip trailing punctuation, a leading article, a trailing parenthetical) rescued 2 more, leaving 7 genuinely unresolvable because they are answer *descriptions* rather than article titles (e.g. "Used furniture dealer", "The first space burial"). We deliberately did not fall back to querying individual words of a multi-word answer, since that risks resolving to an unrelated page and polluting the shared corpus for every other question, not just the one we failed to resolve. Each page is split into paragraphs (boilerplate section headers and fragments under 80 characters dropped), giving a final corpus of **7,660 paragraphs across 284 pages**.

Retrieval. We tokenize with a simple lowercase regex tokenizer and index the corpus with BM25 (`rank_bm25`, $k_1 = 1.5$, $b = 0.75$), retrieving the top-10 passages per question (we keep 10 to compute recall@10; the spec's top-$k = 5$ is used downstream for the answer extractor).

Answering, with no generation. From the top-1 retrieved passage we pick the sentence with the highest lexical overlap with the question (content-word Jaccard overlap, stopwords removed), then extract the *longest* noun-phrase or number span from that sentence, using NLTK POS tagging and a small chunk grammar. No language model is involved anywhere in Task 1.

A bug we found and fixed. Our first chunk grammar allowed a noun phrase to keep absorbing `preposition + another noun phrase` indefinitely, so a phrase like "an adversary *of* the superhero Batman" matched as one giant chunk. Since the extractor picks the *longest* candidate, and almost any sentence has at least one such chain, the extractor was systematically grabbing long generic descriptive phrases instead of the actual (usually short) answer. We restricted the grammar to a standard base noun phrase with no prepositional chaining (`NP: {<DT>?<JJ.*>*<NN.*>+}`), which raised EM from 0.33% to 2% — still low, for reasons that are a genuine property of the heuristic itself and not a further bug (see Section 2.1.3).

2.1.2 Quantitative Results

Table 1: Task 1 results, $n = 300$.

Metric	EM	F1	recall@1	recall@5	recall@10
Task 1 (BM25 + heuristic)	0.020	0.093	0.533	0.717	0.743

Full recall@k for $k \in \{1, 3, 5, 10\}$: 0.533 / 0.660 / 0.717 / 0.743.

Retrieval alone is a reasonably strong classical baseline: BM25 finds a passage containing

the gold answer string 53% of the time at rank 1, rising to 74% by rank 10, despite the index being built from scratch with no bundled evidence. The gap between that and an EM of 2% is entirely a property of the answering step, not retrieval.

2.1.3 Qualitative Analysis

The extractor works exactly when the correct answer happens to be the longest noun-phrase candidate in the chosen sentence, and fails otherwise — it has no notion of relevance, only of phrase length. Correct examples: "Oswald Chesterfield Cobblepot" (Q: *By what name is comic book villain Oswald Chesterfield Cobblepot better known?*), "The Pacific Ocean", "Leslie Lynch King Jr." (Q: *Which U.S. president was born Leslie Lynch King Jr.?*) — in each case the full named entity was the longest chunk available. Failures look like: for *Which World War 2 American general was known as 'Vinegar Joe'?* (gold: Stilwell), the top-1 retrieved document was *Joe Louis* (BM25 matched on "Joe") and the extractor produced "a black orphan boy living" — a longer, completely irrelevant noun phrase from the wrong article. This is the expected, deliberately crude behaviour of a heuristic extractor with no semantic understanding, and it is exactly the gap a generative reader should close in Task 2.

2.1.4 What Did Not Work / Limitations

Besides the chunk-grammar bug already described, the one standing limitation is the 7 unresolvable topics noted above: those questions' gold passage is legitimately absent from the index, so recall@k is correctly 0 for them rather than hidden. We chose not to paper over this by relaxing the resolution strategy, since doing so would have introduced noise into the shared corpus for every other question.

2.2 23CS10008 — Bala Dharnesh

2.2.1 Method

We also build our own Wikipedia corpus from scratch, by a question-only discovery procedure: instead of resolving the gold *answer* to a page title, we search Wikipedia using only the *question* text (via the MediaWiki `search` and `prefixsearch` endpoints, up to 3 pages per question), so no gold answer or alias is ever used to choose which pages enter the index — a stricter leakage safeguard than resolving the answer string directly. Every query, selected title, and question-coverage decision is logged to a discovery manifest, and the corpus-building step retries indefinitely rather than silently skipping a question, reaching 300/300 question coverage with zero zero-result discoveries. Retrieval and answering otherwise follow the assignment's brief directly: BM25 ($k_1 = 1.5$, $b = 0.75$) over the pooled corpus, then the top-1 passage's highest-overlap sentence, then its longest noun phrase or numeric span (NLTK POS tagging and a regex chunk grammar). The implementation is organised as a small installable Python package (`code/task1/{io,bm25,text,build_corpus,build_index,run}.py`) with a dedicated `requirements-task1.txt` and unit tests, rather than a notebook.

2.2.2 Quantitative Results

Table 2: Task 1 results, $n = 300$, independently recomputed from saved predictions by `audit_results.py`.

Metric	EM	F1	recall@1	recall@5	recall@10
Task 1 (BM25 + heuristic)	0.037	0.095	0.287	0.420	0.477

Full recall@k for $k \in \{1, 3, 5, 10\}$: 0.287 / 0.380 / 0.420 / 0.477.

Retrieval reaches a gold-answer-containing passage at rank 1 for 29% of questions, rising to 48% by rank 10, which is respectable given the corpus is built purely from question text with no gold answer ever touching the index. EM and F1 stay low (3.7% and 9.5%), confirming once again that the longest-noun-phrase heuristic, not retrieval, is the bottleneck in this classical pipeline. See Section 2.4 for how this compares with the other two implementations.

2.3 22EE38015 — Subham Ghosh

2.3.1 Method

We build our own from-scratch Wikipedia corpus and follow the assignment's classical pipeline directly: BM25 (`rank_bm25`, $k_1 = 1.5$, $b = 0.75$), highest-overlap-sentence selection, then the longest NLTK-tagged noun phrase or numeric span, with explicit tie-breaking rules (equal-length spans prefer character length, then earliest position). For corpus construction, every question is turned into a stop-word-free keyword query via spaCy, with named entities, noun phrases, and finally the first three keywords as deterministic fallbacks if the first query returns nothing; the first main-namespace Wikipedia result is kept, and full article HTML (not just the API's summary extract) is parsed into non-overlapping 20–120-word paragraphs. This gives a corpus of **297 articles, 15,423 passages**, with zero questions returning no search results and a post-hoc gold-alias coverage of 79.00% (computed after the fact, never used to drop hard questions or add answer-targeted pages).

2.3.2 Quantitative Results

Table 3: Task 1 results, $n = 300$.

Metric	EM	F1	recall@1	recall@5	recall@10
Task 1 (BM25 + heuristic)	0.000	0.050	0.260	0.427	0.480

Full recall@k for $k \in \{1, 3, 5, 10\}$: 0.260 / 0.367 / 0.427 / 0.480.

EM lands at exactly 0 and F1 at 5.0%, with recall@k ranging from 26% at rank 1 to 48% at rank 10 — the same story as every other classical pipeline in this report: retrieval finds a usable passage a meaningful fraction of the time, but the longest-noun-phrase heuristic almost never extracts the actual answer from it. See Section 2.4 for how this compares with the other two implementations.

2.4 Comparison Across the Three Task 1 Implementations

Table 4: Task 1, all three submissions, $n = 300$ each. All three build their own from-scratch Wikipedia corpus and share the same BM25 + heuristic-extraction architecture; only the corpus-construction strategy and incidental implementation choices differ.

Submission (corpus strategy)	EM	F1	R@1	R@5	R@10
23CS30011 (answer-string)	0.020	0.093	0.533	0.717	0.743
23CS10008 (question-only)	0.037	0.095	0.287	0.420	0.477
22EE38015 (question-only)	0.000	0.050	0.260	0.427	0.480

Two things are consistent across all three independent implementations. First, EM is near-zero everywhere (0.0–3.7%) and F1 stays under 10%: the classical extractor's weakness is not an artifact of any one corpus or codebase, it is a structural property of picking "the longest noun phrase" with no notion of relevance, and three independent teams converged on the same failure mode by three different routes. Second, retrieval quality tracks the corpus-construction strategy, not the implementation: our answer-string discovery (which resolves each question's gold answer directly to a page title) reaches noticeably higher recall@k at every k than either question-only discovery pipeline (Bala's and Subham's), which is the expected trade-off — using the answer to help build the corpus is an easier retrieval problem to then solve, at the cost of a less strict (though still never retrieval- or answer-exposing at *scoring* time) corpus- construction procedure. Bala's and Subham's numbers, built independently from different code and different query strategies but the same question-only philosophy, agree with each other far more closely than either agrees with ours, which is a useful independent cross-check that this trade-off is real and not an artifact of one specific implementation.

3 Task 2 — Paper Reproduction: Retrieval-Augmented Generation

3.1 23CS30011 — B. Siddaarth

3.1.1 Method

We reproduce the *idea* of Lewis et al.'s RAG [2] — retrieve, then generate conditioned on what was retrieved — using the retrieval index and 300-question sample from Task 1, on the architecture and settings the assignment specifies:

- **Retriever:** `sentence-transformers/all-MiniLM-L6-v2`, encoding all 7,660 paragraphs and all 300 questions, indexed with FAISS `IndexFlatIP` over L2-normalised vectors (inner product on unit vectors is exact cosine similarity, not an approximation), top-$k = 5$.
- **Generator:** `Qwen/Qwen2.5-7B-Instruct`, loaded in 4-bit (`bitsandbytes`, NF4) on a Colab T4 GPU. Each of the top-5 passages is truncated to 200 tokens using the generator's own tokenizer before being concatenated into the prompt with the question; decoding is greedy (`temperature=0`, 64 max new tokens), with no fine-tuning anywhere — every variant below is a purely in-context use of the model, with no gradient ever touching its weights.

On top of this shared retrieval pipeline, we tried two iterations of the generator's prompt and evaluate both, rather than committing to one upfront:

- **Iteration 1 — zero-shot.** The prompt the assignment's step-by-step instructions literally describe: the top-5 passages plus the question, nothing else.
- **Iteration 2 — few-shot.** We noticed the dataset specification separately notes that TriviaQA's `train` split exists as "a source pool for few-shot exemplars only, never used for training," and treated this as an invitation to test whether in-context demonstrations improve on Iteration 1. We draw three (question, answer) pairs from `train` (seed 42, answers under 20 characters so the exemplars model a visibly concise answer style) and prepend them to the chat history as alternating user/assistant turns before the real question and passages. Retrieval, passages, and decoding settings are identical to Iteration 1 — the exemplars are the only vari-

able, and no weights are touched here either. This runs in a separate notebook, `task2_rag_fewshot.ipynb`, so both iterations are independently reproducible and their outputs never overwrite each other.

Deviations from the paper, explicitly justified. We are not attempting a faithful reproduction of the paper's trained system, and the gap this leaves is discussed in Section 3.1.4. Concretely:

- *Retriever:* the paper uses DPR [3], a BERT bi-encoder trained with retrieval supervision on NQ and TriviaQA; we use an off-the-shelf sentence embedding model with no retrieval-specific training, because training a retriever was out of scope for this assignment and the spec names MiniLM directly.
- *Index:* the paper indexes a full December-2018 Wikipedia dump split into 21M disjoint 100-word chunks; our index is the 7,660-paragraph corpus built in Task 1, covering only the pages relevant to our 300 questions. This is a dramatically easier retrieval problem and the single biggest reason our numbers are not comparable to the paper's.
- *Generator:* the paper trains BART-large (400M parameters) end-to-end jointly with the retriever, marginalising over the top-k latent documents. We use Qwen2.5-7B-Instruct (17.5× more parameters, pretrained far more recently) purely via prompting, with no joint training and no gradient ever flowing through the retriever. This is the deviation the assignment explicitly allows for ("reduced model size due to compute constraints" applies in the opposite direction here — we use a *larger*, more capable but untrained generator because training one was infeasible on a CPU-only/Colab-free-tier budget).

3.1.2 Quantitative Results

Table 5: Task 1 vs. both Task 2 prompting iterations, $n = 300$. Recall@k is identical across Iterations 1 and 2 since only the generator's prompt changes between them; Iteration 2 is our final, best-performing Task 2 result.

System	EM	F1	recall@1	recall@5	recall@10
Task 1 (BM25 + heuristic)	0.020	0.093	0.533	0.717	0.743
Task 2, Iteration 1 (zero-shot)	0.660	0.698	0.513	0.670	0.730
Task 2, Iteration 2 (few-shot)	**0.700**	**0.740**	0.513	0.670	0.730

Three things stand out. First, EM jumps from 2% to 66–70% while recall@5 barely changes (0.717 vs 0.670) — almost the entire gain comes from the generator actually reading and reasoning over the retrieved evidence, not from better retrieval. Second, dense retrieval alone is *slightly worse* than Task 1's BM25 at every k (recall@1: 0.513 vs 0.533; recall@5: 0.670 vs 0.717). This is expected: MiniLM is a general-purpose sentence embedder, not fine-tuned for retrieval like DPR, and our corpus is entity-heavy trivia content where exact keyword overlap — BM25's strength — is a strong signal. This observation is what motivates fusing the two signals in Task 3. Third, moving from Iteration 1 to Iteration 2 raises EM by 4 points and F1 by 4.2 points, with recall@k completely unchanged at every k by construction, since the three few-shot exemplars never touch retrieval — the entire gain is attributable to the generator producing better-formatted, more reliably concise answers once shown worked examples, which Section 3.3

discusses concretely. We report **Iteration 2 as Task 2's final result** going forward in this report, since it is the better-performing of the two generator-prompting strategies we actually tested, and keep Iteration 1 alongside it as the baseline that motivated trying a second iteration in the first place.

3.1.3 Qualitative Analysis

Most correct generations are unremarkable in the best way — the model reads the right passage and states the answer concisely ("Lauren Bacall", "Pittsburgh", "Helium", "28"). One example is worth dwelling on: for *What consumer electronics product shares its name with the common name of the fruit Rubus Fructicosis?* the model answered correctly, "BlackBerry" – but its top-1 retrieved passage was about **tartaric acid**, completely unrelated to the question. The model did not use the retrieved evidence at all; it answered from its own pretrained world knowledge. This is a direct instance of the phenomenon the RAG paper itself reports (§4.1: RAG can answer correctly even when no retrieved document contains the answer, achieving 11.8% accuracy on such NQ cases where an extractive model would score 0%) and is exactly the failure mode Task 3's groundedness check is built to detect. A second recurring failure mode in Iteration 1 is instruction-following slippage: for *"Because I'm worth it" was the advertising slogan for which product on TV?* (gold: L'Oréal), Iteration 1 produced *"Passage information does not match the question. No relevant passage provided."* — a hedge, not a guess, which the spec's prompt explicitly asked it not to produce, and which costs F1 even on questions where a short answer was available.

Iteration 1 vs. Iteration 2, concretely. Comparing both iterations' predictions question-by-question, few-shot prompting fixes 15 of Iteration 1's errors and introduces 3 new ones, a net +12/300 that accounts for the full EM gain in Table 2. The same "Because I'm worth it" question above is one of the fixes: shown three worked examples of a short, direct answer format, Iteration 2 abandons the hedge and answers *"L'Oréal"* correctly, using the same retrieved passages Iteration 1 had already seen. Other fixes are similar formatting corrections rather than new reasoning — *"Hawaiian Islands"* tightened to *"Hawaii"*, *"Para Handy's"* tightened to *"Para Handy"* — which supports reading the gain as better compliance with the requested answer format, not a change in what the model actually knows. The 3 regressions run the other way: Iteration 2 occasionally adds detail an already-correct answer did not need ("Barings" → "Barings LLC") or substitutes a near-synonym ("Complain" → "Grumble") that the exact-match scorer does not credit, even though both are reasonable answers to a human reader.

3.1.4 Paper-Reproduction Comparison

Table 6: EM comparison. The right column is as reported in the paper; our two iterations are shown as fractions for consistency with the rest of this report.

	Our Task 2 ($n = 300$)	**Lewis et al. 2020, Table 1 (RAG-Sequence)**
EM, Iteration 1 (zero-shot)	0.660	56.8 (open-domain test) / 68.0 (TQA-Wiki test)
EM, Iteration 2 (few-shot, final)	**0.700**	56.8 (open-domain test) / 68.0 (TQA-Wiki test)

Both our iterations land numerically close to — Iteration 2 even above — the paper's open-domain figure (56.8%), but this is **not a meaningful comparison in our favour**, for reasons already laid out in the method section: our retrieval problem is far easier (284 relevant pages vs. 21M Wikipedia chunks), and our generator is a modern 7B instruction-tuned model used purely via prompting rather than a 400M model trained specifically to use retrieved evidence. The "BlackBerry" example above makes this concrete: a meaningful fraction of our correct answers are not actually grounded in retrieval at all, they come from the generator's own knowledge, which a smaller, untrained-era model like BART-large simply would not have had as much of. The honest reading of this comparison is that it isolates how much of a RAG system's apparent performance, under compute-constrained reproduction, comes from the generator's raw capability versus the retrieval-augmentation mechanism itself — and in our setup, a large share of it is the former, regardless of which of our two prompting iterations is used.

3.2 23CS10008 — Bala Dharnesh

3.2.1 Method

We reuse our Task 1 corpus and swap BM25 for dense retrieval: `sentence-transformers/all-MiniLM-L6-v2` embeddings over the full paragraph set, indexed in FAISS and searched with cosine similarity, top-5. Generation uses `Qwen/Qwen2.5-1.5B-Instruct` loaded in 4-bit NF4, each passage truncated to 200 generator tokens, greedy decoding with 64 max new tokens — the architectural brief the assignment specifies, with a generator sized to fit an RTX 3050's 4GB of VRAM, tested end to end in Docker rather than a notebook. Both the dataset/model checkpoints are pinned to immutable HuggingFace revisions, documented in `configs/task2.json`.

A prompt-wording fix. Our first Task 2 run instructed Qwen only to give a "short answer," which inspection showed sometimes produced a full explanatory sentence instead of a short span. We tightened the instruction to return only the shortest answer phrase (deliberately still allowing multi-word answers, since 149 of the 300 canonical TriviaQA answers are multi-word), changing nothing else — same corpus, checkpoints, retrieval, and decoding settings. This raised EM from 25.67% to 32.67% and F1 from 32.53% to 37.99%, with recall@5 unchanged at 43.67%, confirming the gain is purely a generation-formatting effect, not a retrieval change.

3.2.2 Quantitative Results

Table 7: Task 1 vs. Task 2, $n = 300$.

System	EM	F1	R@1	R@3	R@5
Task 1 (BM25 + heuristic)	0.037	0.095	0.287	0.380	0.420
Task 2 (MiniLM + Qwen2.5-1.5B)	**0.327**	**0.380**	0.247	0.400	0.437

EM jumps by roughly 29 points once a generator reads the retrieved evidence instead of a longest-noun-phrase heuristic, while recall@k is actually slightly *lower* for the dense retriever than for BM25 at every reported k (recall@1: 0.247 vs. 0.287; recall@5: 0.437 vs. 0.420 is the one exception, by a small margin) — a general-purpose sentence embedder with no retrieval-specific fine-tuning does not reliably beat BM25 on this entity-heavy, trivia-style corpus. See Section 3.4 for how this compares with the other two implementations.

3.3 22EE38015 — Subham Ghosh

3.3.1 Method

We follow the same brief with two notable choices: the encoder is `all-MiniLM-L6-v2`, indexed with an exact (non-approximate) `faiss.IndexFlatIP` over L2-normalised vectors so that inner product is exact cosine similarity; the generator is a smaller, CPU-friendly `Qwen/Qwen2.5-0.5B-Instruct`, explicitly chosen so the whole pipeline can run without a GPU (the reference run is CPU PyTorch on Linux/Python 3.12). The prompt asks for the shortest answer phrase, permits an explicit `UNKNOWN` token if the evidence is insufficient (converted to an empty prediction rather than a hedge sentence), and decoding is greedy with at most 64 new tokens — a frozen generator used purely in-context, with no fine-tuning anywhere.

3.3.2 Quantitative Results

Table 8: Task 1 vs. Task 2, $n = 300$. Recall@k is the answer-containing-passage proxy.

System	EM	F1	R@1	R@3	R@5
Task 1 (BM25 + heuristic)	0.000	0.050	0.260	0.367	0.427
Task 2 (MiniLM + Qwen2.5-0.5B)	**0.193**	**0.264**	0.300	0.407	0.453

EM rises sharply once more from Task 1 to Task 2 (0.0% → 19.3%), the same pattern seen throughout this report, though more modestly than with a larger generator (see Section 3.4). Dense retrieval here is slightly *better* than BM25 at every k (recall@1: 0.300 vs. 0.260; recall@5: 0.453 vs. 0.427), unlike the other two implementations in this report, where dense retrieval trails BM25 instead.

3.3.3 Qualitative Analysis

A correct case worth noting: for *Which one of the seven member territories is also capital of UAE (United Arab Emirates)?* the top passage explicitly calls Abu Dhabi the national capital, and the generated answer matches it exactly, directly supported by the retrieved evidence. A more interesting incorrect case shows the limits of exact-match scoring rather than a generation failure: for *What are fairy rings made of?* (gold: *Mushrooms*), the top passage explicitly says fairy rings are rings of mushrooms, but the model answers *Fungi* — a broader, not-unreasonable term that scores EM=0 and F1=0.67 even though the evidence was read correctly, with the unhandled *fungus/fungi* morphological change also driving its lexical overlap score to zero.

3.4 Comparison Across the Three Task 2 Implementations

Table 9: Task 2, all three submissions, $n = 300$ each. All three share the same architecture (frozen dense retriever + frozen generator, no fine-tuning, greedy decoding) and differ mainly in generator size and the retrieval corpus inherited from each submission's own Task 1.

Submission	Generator	EM	F1	recall@5
23CS30011	Qwen2.5-7B-Instruct	**0.700**	**0.740**	0.670
23CS10008	Qwen2.5-1.5B-Instruct	0.327	0.380	0.437
22EE38015	Qwen2.5-0.5B-Instruct	0.193	0.264	**0.453**

The clearest pattern across all three independent implementations is a strong, monotonic relationship between generator size and EM/F1 (7B > 1.5B > 0.5B, in that order, exactly matching parameter count) despite three different retrieval corpora and three different exact prompt templates — evidence that, in this compute-constrained, no-fine-tuning setting, the generator's raw capability is doing most of the work, which is the same conclusion our own paper-reproduction comparison (Section 3.1.4) reaches from a different angle. Recall@5, by contrast, does *not* follow the same ordering (22EE38015's dense retriever is actually best by this measure despite having by far the weakest EM), which reinforces a point we make repeatedly in this report: retrieval quality and final-answer quality are only loosely coupled once a generator is in the loop, and the relationship between the two is corpus- and generator-dependent rather than a fixed property of "dense retrieval" in general.

4 Task 3 — Combine & Improve: Hybrid Retrieval with Groundedness Verification

4.1 23CS30011 — B. Siddaarth

4.1.1 Method

Task 3 asks for two distinct combinations, and we implement both.

1. Fusing the retrieval signals. We rebuild Task 1's BM25 ranking and Task 2's dense ranking (same settings as before, top-10 each) and combine them with Reciprocal Rank Fusion, constant = 60:

$$\mathrm{RRF}(d) = \sum_{\text{ranker} \in \{\text{BM25,dense}\}} \frac{1}{60 + \mathrm{rank}_{\text{ranker}}(d)}$$

summed only over rankers in which passage d appears, giving a fused top-10 (and top-5) per question.

2. Fusing Task 1's idea with Task 2's outputs. We generate with the hybrid top-5 passages using the exact same generator and settings as Task 2 — the *only* change from Task 2 is which passages are fed in, which isolates the effect of retrieval fusion cleanly from any change in generation. As in Task 2, we tried two iterations of the generator's prompt on top of this hybrid retrieval, using the same exemplars in both tasks:

- **Iteration 1 — zero-shot.** Qwen2.5-7B-Instruct, 4-bit, greedy decoding, 64 max new tokens, prompted with the hybrid top-5 passages plus the question — the assignment's literal prompt description.
- **Iteration 2 — few-shot.** The identical three `train`-split exemplars from Section 3 (Task 2), prepended to the same generator's chat history, with everything else unchanged. This runs in a separate notebook, `task3_hybrid_groundedness_fewshot.ipynb`, so both iterations stay independently reproducible.

We call the non-retried output of each iteration **pass-1** and, as in Task 2, report **Iteration 2 (few-shot) as Task 3's final result**, since it is the better-performing of the two we tested; Iteration 1 is kept as the baseline. We then reuse Task 1's lexical-overlap idea, applied this time to Task 2-style generations: for each generated answer, we strip stopwords and compute what fraction of its remaining content words appear in the concatenated text of its retrieved passages. Answers below 0.5 overlap are flagged **ungrounded** – likely

not actually supported by what was retrieved. We ran the groundedness check and the two correction experiments below for both iterations; the results and conclusions are the same in both cases, so the detailed qualitative discussion uses Iteration 1's generations (where the analysis was done first) and we report Iteration 2's headline numbers alongside it.

Two correction experiments, both net negative. We report both honestly rather than discard them. The assignment's objective asks us to "flag (or abstain on)" ungrounded answers; we went further and tested whether the flag could be used to actively *fix* answers, which is the "Improve" the task title invites:

- **Attempt A — Task-1-style fallback:** replace flagged answers with Task 1's heuristic extraction on the hybrid top-1 passage.
- **Attempt B — LLM retry:** re-prompt the *same* generator with an expanded top-10 context and an explicit instruction to answer `UNKNOWN` if the answer is genuinely not present, rather than guess.

Both are described fully, with why they failed, in Section 4.1.4. Because neither improved results, **the headline Task 3 numbers we report are pass-1**, with the groundedness check used purely as the diagnostic the spec's point 4 asks for, not as an answer editor.

4.1.2 Quantitative Results

Table 10: Retrieval-only comparison, $n = 300$. Hybrid RRF beats both individual rankers at $k = 1, 3, 10$ and is within 0.004 of BM25's recall@5 — exactly the result RRF is meant to produce.

Retriever	recall@1	recall@3	recall@5	recall@10
BM25 (Task 1)	0.533	0.660	**0.717**	0.743
Dense (Task 2)	0.513	0.620	0.670	0.730
Hybrid RRF (Task 3)	**0.547**	**0.663**	0.713	**0.780**

Table 11: End-to-end comparison, $n = 300$, pass-1 (no retry). Within each task, Iteration 2 (few-shot) is the better-performing and final-reported result; Task 3's gain over Task 2 at matched iterations (Iteration 1: EM +0.010; Iteration 2: EM +0.000, F1 −0.001) is small and attributable to retrieval fusion, since the generator and prompt are identical to Task 2 within each iteration.

System	EM	F1
Task 1 (BM25 + heuristic)	0.020	0.093
Task 2, Iteration 1 (zero-shot)	0.660	0.698
Task 2, Iteration 2 (few-shot, final)	0.700	0.740
Task 3, Iteration 1 (zero-shot)	0.670	0.711
Task 3, Iteration 2 (few-shot, final)	**0.700**	**0.739**

Groundedness check, as a diagnostic (spec point 4). Under Iteration 1, of the 201 correct pass-1 answers 91.5% are correctly flagged grounded, and of the 99 incorrect answers 48.5% are correctly flagged ungrounded, giving the flag an overall correctness-prediction accuracy of 77.3%. Under Iteration 2 the pattern holds (89.0% / 38.9% /

74.0% respectively) — slightly lower across the board, consistent with few-shot prompting producing more varied phrasing that the lexical-overlap check is somewhat less tuned to, but the check remains clearly informative in both iterations. This is useful but imperfect in both directions, discussed next.

4.1.3 Qualitative Analysis

The hybrid retriever's wins are typically close calls where BM25 and dense retrieval individually missed by one or two ranks but agreed closely enough on *something* relevant that RRF pulled it into the top ranks. The groundedness check correctly identifies cases like the Task 2 "BlackBerry" example from a different angle: when we ran it over pass-1's hybrid-retrieved output, a parallel case appeared for *In April 1963, Lester Bowles Pearson became Prime Minister of which country?* (gold: Canada). The model answered "Canada" correctly, but all five retrieved passages (documents titled "1957", "1957", "1919", "Iceland", "Batley") were completely unrelated to the question — retrieval failed outright, and the model fell back on background knowledge. The overlap score here was 0.0, and the flag (*ungrounded*) was technically correct about the lack of supporting evidence, even though the answer itself was right.

Iteration 1 vs. Iteration 2 on the hybrid pipeline. As in Task 2, few-shot prompting fixes more pass-1 errors than it introduces: 11 fixed against 2 broken, net +9/300, matching the EM gap in Table 4. The clearest fix is the same hedging pattern seen in Task 2: for *Which US property tycoon bought Turnberry Golf Course in April?* (gold: Donald Trump), Iteration 1 answered *"Not provided in the passages."*, while Iteration 2, given the same hybrid-retrieved passages, answered *"Donald Trump"* correctly. This reinforces the Task 2 reading that few-shot's gain here is about the generator committing to a direct answer more reliably, not about seeing different or better evidence.

Progression across all three tasks. Table 12 tracks a handful of questions through all three systems. The common pattern is Task 1 failing in the way described in Section 2.3 (longest-noun-phrase, not relevance), while both Task 2 and Task 3 succeed once a generator is involved. The more interesting rows are the last two: *"Because I'm worth it"...* and *John Crome...*, where Task 2's dense-only retrieval surfaced an irrelevant or borderline passage and the model either refused to answer or guessed wrong ("Dunwich School" instead of "Norwich School"), while Task 3's hybrid retrieval found the right page and the *same* generator then answered correctly. These are the concrete instances behind Task 3's 0.010 EM gain over Task 2 — small in aggregate, but traceable to specific, explicable retrieval wins.

Table 12: A question tracked through all three systems. Task 1's answers are the longest noun phrase in the wrong sentence; Task 2 and Task 3 both involve the generator, with Task 3's hybrid retrieval resolving two cases Task 2 got wrong.

Question (gold)	**Task 1**	**Task 2**	**Task 3**
Cagliari is the capital of which island? (Sardinia)	the Piedmontese officers	Sardinia	Sardinia
Who was the first Astronomer Royal? (John Flamsteed)	the first Astronomer Royal	John Flamsteed	John Flamsteed
"Because I'm worth it" was the slogan for which product? (L'Oréal)	30-year-old McCann copywriter Ilon Specht	"Passage information does not match the question..."	L'Oréal
John Crome was the main artist of which group of English painters? (Norwich School)	Satoshi Tajiri	Dunwich School	Norwich School

4.1.4 Why the Correction Experiments Failed

Attempt A (Task-1-style fallback) replaced every flagged answer with Task 1's heuristic extraction on the top-1 passage. Result: **0 questions fixed, 17 questions broken**, net −17/300 EM. The reason is simple in hindsight: Task 1's extractor only reaches 2% EM on its own, so swapping a 66–70%-accurate generator's answer for it is a downgrade almost by definition, and 8.5% of genuinely correct pass-1 answers get incorrectly flagged ungrounded (the Lester Pearson pattern above) and are destroyed by the swap for essentially no compensating gain.

Attempt B (LLM retry with top-10 + "say UNKNOWN") used the same strong generator instead of a weak heuristic, which we expected to be safer. Result: **1 fixed, 14 broken**, net −13/300 — better than Attempt A, but still net negative, and for an identifiable, structural reason rather than a weaker method. Inspecting the 14 broken cases shows every one was *already correct* in pass-1 ("28", "Canada", "Illinois", "Australia", "Japan", "London", …) but got flagged ungrounded because retrieval genuinely failed to surface anything relevant — the same pattern as the Lester Pearson example. When we then tell the model "your previous answer did not clearly match the evidence, reconsider, and say UNKNOWN if you cannot find it," it second-guesses an answer it actually had right and discards it, because widening the context from top-5 to top-10 cannot retrieve evidence that was never indexed or ranked highly enough to begin with. This mirrors exactly what the RAG paper identifies as a strength of generative QA — the ability to answer correctly from parametric knowledge even when retrieval fails (§4.1) — and shows the other side of that coin: *any* correction strategy that penalises "correct but ungrounded" answers will lose points on precisely the cases where that strength saved a failed retrieval, and this is a property of the failure mode, not of the specific correction method we chose.

We also found a secondary, narrower flaw in the overlap metric itself while investigating a different failure: for *What percentage of the earth's surface is covered by Europe?* (gold: 8%), the model answered `2.5%` — wrong, and from an unrelated top-1 passage ("Pacific

Ocean") — yet the overlap score was 1.0 ("grounded"). Our tokenizer splits `"2.5%"` on the decimal point into two single-digit tokens, `"2"` and `"5"`, and lone digits are common enough across five pooled passages that a false match is almost guaranteed by chance. This affects short numeric answers specifically and contributes to the 48.5%-ungrounded-detection rate on incorrect answers being well under 100%; it does not, however, explain the correction experiments' failures above, which are a retrieval-coverage problem rather than a tokenization artifact.

The same correction experiments, repeated under Iteration 2. We re-ran the retry experiment (Attempt B) on Iteration 2's pass-1 output, to check whether few-shot prompting – which already fixes some hedging failures on its own – changes this conclusion. It does not: **2 fixed, 17 broken**, net −15/300, the same net-negative pattern as Iteration 1, for the same structural reason. Few-shot prompting improves answer *formatting*, not retrieval coverage, so it does not change which questions have usable evidence in the top-k passages – and retry still penalises the correct-but-ungrounded answers that a failed retrieval, not a weak generator, produced.

Conclusion for Task 3. The headline, validated improvement is the retrieval fusion (Section 4.2) — real, modest, and attributable to a clean mechanism, and it holds under both prompting iterations. Few-shot prompting (Iteration 2) is a further, independent improvement on top of it, raising Task 3's pass-1 EM from 0.670 to 0.700, which is why we report it as Task 3's final result (Table 4). The groundedness check is a genuinely informative diagnostic (77–74% accuracy at predicting correctness, depending on iteration) but should not be used to overwrite answers with either a weaker method or a second LLM call under current conditions, because its errors are concentrated exactly where correction is most likely to do harm: on answers the model got right despite a failed retrieval – a pattern that persists regardless of how the generator is prompted.

4.2 23CS10008 — Bala Dharnesh

4.2.1 Method

We fuse our own Task 1 BM25 ranking and Task 2 dense ranking with reciprocal rank fusion (constant 60, same formula and candidate depth as elsewhere in this report), feeding the fused top-5 to the identical Task 2 generator and prompt so that retrieval fusion is isolated from any change in generation. Our groundedness check marks an answer **supported** only when its complete normalized phrase occurs in at least one of the same five truncated passages shown to the generator, and flags unsupported answers without silently replacing them.

4.2.2 Quantitative Results

Table 13: Task 2 vs. Task 3, $n = 300$.

System	EM	F1	recall@1	recall@3	recall@5
Task 2 (dense RAG)	0.327	0.380	0.247	0.400	0.437
Task 3 (hybrid RRF + RAG)	**0.343**	**0.404**	**0.303**	**0.420**	**0.460**

Hybrid fusion beats dense-only retrieval at *every* reported metric here (EM +1.66pp, F1 +2.43pp, recall@5 +2.3pp) — a clean, consistent win across the board. See Section 4.4 for how this compares with the other two implementations.

Table 14: Groundedness confusion matrix, $n = 300$.

Gold answer outcome	Supported	Flagged
Exactly correct	88	15
Incorrect	144	53

Groundedness-check accuracy (treating "supported" as a prediction of correctness and "flagged" as a prediction of incorrectness): **47.00%**. Decomposed, 85.44% of correct answers have high lexical overlap while only 26.90% of incorrect answers have low overlap, so the check is considerably more conservative about flagging wrong answers than the implementations elsewhere in this report, though the underlying conclusion — a useful warning signal, not a correctness proof — holds just the same.

4.3 22EE38015 — Subham Ghosh

4.3.1 Method

We follow the same brief again: BM25 and dense search each supply their top-50 candidates, reciprocal rank fusion with constant 60 produces the fused top-5, fed to the identical Task 2 generator, prompt, truncation, and decoding settings. Our groundedness checker computes token-multiset overlap between the normalized predicted answer and the exact (truncated) evidence text the generator saw, flagging anything below a fixed threshold of 0.8. We additionally run a small, fixed-seed pilot of independent *support* labels (ten questions, evidence-conditioned judgments distinct from answer-string EM) precisely to probe where lexical overlap and genuine evidentiary support disagree.

4.3.2 Quantitative Results

Table 15: Task 2 vs. Task 3, $n = 300$.

System	EM	F1	recall@1	recall@3	recall@5
Task 2 (dense RAG)	0.193	0.264	0.300	0.407	0.453
Task 3 (hybrid RRF + RAG)	**0.197**	**0.286**	**0.310**	**0.417**	**0.463**

Fusion's gain here is small in absolute EM (+0.33pp) but positive at every reported metric. The churn behind that small net number is worth stating directly: hybrid retrieval corrects 13 of dense retrieval's errors while losing 12 previously-correct answers, with both systems agreeing on 46 questions — a large-churn/small-net-gain pattern, and one worth keeping in mind whenever a net number alone is reported without the churn behind it.

Table 16: Groundedness statistics (dense RAG vs. hybrid RAG).

Statistic	Dense RAG	Hybrid RAG
Correct answers with high overlap	100.00%	94.92%
Incorrect answers with low overlap	39.26%	29.88%
Support-check accuracy (nonempty pilot audit)	50.00%	55.56%

4.3.3 Qualitative Analysis

A sharp illustration of exactly the failure mode our own groundedness discussion centres on (Section 4.1.4) turns up in a reviewed dense-RAG case: the question asks which country contains Masvingo, Midlands, and Manicaland, and the model answers *Mozambique*, although the evidence places Masvingo in Zimbabwe and names Mozambique only as a neighbouring country — a *high-overlap, high-confidence, wrong* answer, the exact inverse of the failure pattern a lexical-overlap checker is built to catch. The pilot support audit makes the same point we make from a different angle: lexical overlap and evidentiary correctness are related but genuinely separate signals, and conflating them (treating "high overlap" as "verified") would be a mistake in any of the three submissions' pipelines, not just one.

4.4 Comparison Across the Three Task 3 Implementations

Table 17: Task 3, all three submissions, $n = 300$ each. All three implement the same reciprocal-rank- fusion-plus-groundedness-check brief; thresholds and exact overlap definitions differ.

Submission	Task 2 EM → Task 3 EM	Fusion's EM gain	Groundedness accuracy
23CS30011	0.700 → 0.700 (final)	+0.000	74.0% (Iter. 2)
23CS10008	0.327 → 0.343	+0.017	47.0%
22EE38015	0.193 → 0.197	+0.003	overlap-based; not a single accuracy

Three independent implementations agree that reciprocal rank fusion is never harmful and usually mildly helpful at the retrieval level (Section 2.4's recall@k story repeats here), but its effect on final *EM* ranges from essentially zero (ours, once few-shot prompting is already fixing the same errors fusion would have fixed) to a clearly positive 1.7 points (Bala's). This is consistent with a theme that recurs across all three submissions' Task 3 sections: retrieval-side improvements and end-to-end answer quality are only loosely coupled once a generator sits between them, and a fixed retrieval-fusion recipe can land anywhere in that range depending on how much headroom the generator and prompt have already used up. On groundedness, the three submissions use three different operational definitions of "grounded" (whole-phrase containment for us and Bala, 0.8 token-overlap for Subham) and arrive at noticeably different accuracy numbers (74% vs. 47%), which is itself an instructive result: a single number like "groundedness accuracy" is highly sensitive to the exact overlap rule chosen, and should always be read together with its definition rather than compared as if it were a universal metric.

5 Discussion

What three independent implementations agree on. Before discussing any one submission's specific findings in detail, it is worth stating plainly what survived being built three times, by three people, with three different corpora, generators, and codebases, because that is the strongest form of evidence this report can offer. All three submissions found: (1) the classical BM25-plus-heuristic-extraction baseline gets retrieval mostly right (recall@5 between 42% and 72% depending on corpus-construction strategy) but answering

badly wrong (EM between 0% and 3.7%), because "longest noun phrase" has no notion of relevance; (2) swapping that heuristic for a frozen generator reading the retrieved evidence is by far the single largest improvement in the whole project, dwarfing every retrieval-side change, in all three codebases, regardless of which generator size was used; (3) a general-purpose dense embedding model (MiniLM, untouched by retrieval-specific fine-tuning) does not reliably beat BM25 on this entity-heavy, trivia-style corpus, in two of the three submissions, with the third (Subham's) as an instructive partial exception, discussed in Section 3.4; and (4) reciprocal rank fusion of the two retrieval signals is never harmful and usually mildly helpful, in all three submissions, though its translation into end-to-end EM gain varies a great deal (Section 4.4). None of us coordinated on these conclusions in advance — we arrived independently, which is exactly the kind of agreement a single implementation cannot offer on its own.

Where the three submissions genuinely differ. The three Task 1 corpus-construction strategies read Dataset 10.3's missing, underspecified corpus-construction section differently: 23CS30011 resolves each question's *gold answer* to a Wikipedia page title (an easier retrieval problem, since the answer itself anchors the corpus, but with the answer touching corpus construction, never scoring); 23CS10008 and 22EE38015 both instead search Wikipedia using only the *question* text, a stricter leakage safeguard that comes at the cost of measurably lower recall@k for both of them (Section 2.4). Generator size varies by more than an order of magnitude across the three Task 2/3 implementations (0.5B, 1.5B, 7B parameters) and tracks EM almost perfectly (Section 3.4), which is itself a useful finding about how compute-constrained this assignment's RAG reproduction inherently is: a large part of what looks like a "better RAG system" in this setting is actually just "a bigger frozen language model," independent of retrieval quality. The rest of this Discussion focuses on 23CS30011's own pipeline and experiments in more depth, since that is the implementation we can speak to with the most procedural detail (notebook-level iteration history, per-question diffs, and the two correction experiments in Section 4.1.4); the cross-submission comparison tables in each Task section above should be read as the complementary, breadth-over-depth half of this report's evidence.

What worked. The from-scratch Wikipedia index construction in Task 1 worked well enough (97.6% topic resolution, 7,660 usable paragraphs) to support all three tasks without further changes. BM25 is a strong, cheap baseline on this entity-heavy corpus, and RRF fusion delivered exactly the textbook result — a hybrid ranking at least as good as, and usually better than, either input ranking alone (recall@1 0.547 vs. 0.533/0.513; recall@10 0.780 vs. 0.743/0.730). Most importantly, the single largest lever across the whole project was the one the assignment is fundamentally about: swapping a relevance-blind extractive heuristic for a generator that actually reads the retrieved context took EM from 2% to 66–70% (Task 2 and Task 3, across both prompting iterations), which dwarfs every retrieval-side improvement we measured. This is worth stating plainly because it is easy, when building three increasingly elaborate systems, to lose sight of the fact that the biggest single win in the whole project came from the simplest possible change — replacing a relevance-blind heuristic with a model that can read.

What did not work, and why that is still useful. Both attempts to turn the groundedness flag into an automatic correction mechanism failed, and we think this negative result is more informative than a quiet success would have been. It demonstrates a real tension in retrieval-augmented generation: a model's ability to fall back on parametric knowledge when retrieval fails is a genuine strength (exactly as Lewis et al. report

for NQ), but it is indistinguishable, from a lexical-overlap check alone, from a model that is hallucinating over good evidence. Any policy that treats low overlap uniformly as "untrustworthy, replace it" will necessarily sacrifice some of that strength. A more promising direction we did not have time to pursue would be separating these two cases *before* deciding whether to correct — for example, trusting an ungrounded answer more when the retrieval scores themselves are low across the board (suggesting the corpus likely does not contain the answer) and trusting it less when retrieval scores are high but the generated answer still does not overlap with the top passages (suggesting the model ignored good evidence). A second, cheaper idea worth trying is confidence calibration from the generator itself — comparing the log-probability of the greedy answer against the log-probability of an explicit "UNKNOWN" completion — rather than relying only on surface overlap with retrieved text, since the two signals would fail in different, partially independent ways and could be combined.

Two prompting iterations, evaluated on equal footing. For both Task 2 and Task 3 we did not settle for a single prompt and move on: we built and fully evaluated two iterations of the generator's prompt (Iteration 1, zero-shot; Iteration 2, few-shot, using exemplars drawn from the `train` split the dataset specification names for exactly this purpose) and compared them the same way we compared every other design choice in this project, by measuring EM, F1, and recall@k on the full 300-question sample for each. Iteration 2 won in both tasks, by a consistent margin (Task 2: EM +0.040, F1 +0.042; Task 3: EM +0.030, F1 +0.028), with recall@k completely unchanged in both cases, since the exemplars only affect generation. We therefore report **Iteration 2 as the final result for both Task 2 and Task 3** throughout this report, with Iteration 1 retained as the baseline that motivated building a second iteration in the first place and as the reference point for the paper-reproduction comparison and the groundedness diagnostics. Tracing the gain question-by-question (Sections 3.3 and 4.5) shows it is driven by format compliance — Iteration 2 answers more reliably in the short, direct form the task asks for, rather than hedging ("Not provided in the passages.") or padding a correct answer with extra words — rather than by any change in what the model actually knows or which passages it is given. Confirming this, Task 3's retry correction experiment fails under Iteration 2 in essentially the same way it fails under Iteration 1 (Section 4.1.4): few-shot prompting cannot close the retrieval-coverage gap that correction is trying to patch, because it never touches retrieval at all.

On the paper-reproduction comparison. We want to be explicit that Task 2's numerically favourable comparison to Lewis et al. (Section 3.1.4) is not evidence that our pipeline is better than theirs. It reflects a much smaller, easier retrieval corpus and a vastly more capable, more recently pretrained generator used with no training at all, against their jointly-trained, much smaller model over the full Wikipedia dump. We think this comparison is still worth reporting precisely because the gap it reveals is informative: it suggests that in a compute-constrained reproduction, a strong off-the-shelf generator can mask a large fraction of what the retrieval-augmentation mechanism is actually supposed to contribute.

On our evaluation metrics themselves. We found two places where the metrics we built (reasonable, standard choices — SQuAD-style EM/F1, substring-match recall@k) have their own blind spots, and we think these are worth stating rather than quietly working around. First, the "2.5%" tokenization issue described in Section 4.1.4 is a general hazard for any lexical-overlap metric applied to short numeric answers: splitting on

punctuation turns a specific number into generic single digits that match almost anything. Second, substring-match recall@k (does the normalised gold string appear verbatim inside a retrieved passage?) undercounts cases where a passage paraphrases the answer rather than stating it verbatim, so our reported recall numbers are a lower bound on how often the *information* needed was actually retrieved, not an exact figure. Neither issue changes the overall story of this report, but both are exactly the kind of detail that is easy to miss if a metric is treated as ground truth rather than as its own approximation.

On why BM25 held its own against a neural retriever. It is tempting to assume a dense embedding model should beat a 1990s-era lexical scoring function by default, and our results (Table in Section 3.2) say otherwise for this dataset specifically. TriviaQA's questions are trivia-style and name entities directly ("Which World War 2 American general was known as 'Vinegar Joe'?"), so the exact words in the question are usually also the exact words in the relevant passage — precisely where BM25's term-matching is strong and a general-purpose sentence embedder, with no retrieval-specific fine-tuning, has no particular advantage. We would expect the balance to tip further toward dense retrieval on a dataset with more naturally-phrased, search-engine-style queries where the vocabulary of the question and the answer passage diverge more (Natural Questions, one of the other two datasets offered for this assignment, is a closer fit to that profile) — which is itself a useful, dataset-dependent lesson: which retriever "should" win is an empirical question, not a given.

Engineering lessons. A non-trivial part of this project was making a GPU-dependent generator (Qwen2.5-7B-Instruct, used in Tasks 2 and 3) coexist with an otherwise CPU-only local setup. We settled on a single notebook per task that detects whether it is running on Colab or locally: on Colab it uploads the two Task-1 artifacts, runs generation on a free T4 GPU in 4-bit (`bitsandbytes`), and offers the results for download; run locally it reuses an already-downloaded generation file to redo retrieval and scoring without needing a GPU at all. This made the generation step (the one genuinely expensive part of the whole pipeline) reproducible without requiring paid compute, at the cost of a short manual upload/download step between environments — a reasonable trade given the assignment's own allowance for compute-constrained deviations.

What we would do with more time or compute. Three directions stand out. First, replace MiniLM with a retrieval-fine-tuned encoder (even a lightweight one trained via contrastive learning on TriviaQA question–passage pairs) to see how much of the BM25-vs-dense gap in Section 3.2 closes. Second, scale the index beyond the 284 pages built from our 300 questions' answer entities toward something closer to the paper's full-Wikipedia setting, to get a fairer read on the paper-reproduction comparison in Section 3.1.4. Third, revisit the groundedness-correction idea with a less blunt trigger than a single overlap threshold — the discussion above on separating "retrieval genuinely failed" from "model ignored good evidence" is, we think, the right next step rather than a different correction method applied under the same all-or-nothing trigger that caused both of our attempts to fail.

References

[1] Mandar Joshi, Eunsol Choi, Daniel S. Weld, Luke Zettlemoyer. *TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension.* ACL 2017.

[2] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. *Retrieval-Augmented Generation*

for Knowledge-Intensive NLP Tasks. NeurIPS 2020. arXiv:2005.11401.

[3] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, et al. *Dense Passage Retrieval for Open-Domain Question Answering*. EMNLP 2020.